



VIT[®]
BHOPAL

*Education for
a better world*

PROJECT REPORT

StudyTrack

Student Academic Task & Performance Manager

Submitted By:

Yashraj Malviya

Registration Number : 26MIP10068

Class ID : 1337

Course Code : CSE1021

Faculty Name : Devendra Kumar Vedi

Course : M.Tech Integrated

Branch : CSE



INDEX / TABLE OF CONTENTS

	Section	Page
1	Introduction	3
2	Problem Statement	3
3	Objectives	4
4	Functional Requirements	4
5	Non-Functional Requirements	4
6	System Architecture	5
7	Process Flow / Workflow	6
8	UML Diagrams	7
9	Implementation	8
10	Data & Storage Design	9
11	Testing	10
12	Results	10
13	Challenges Faced	11
14	Learnings	11
15	Future Enhancements	11
16	References & Conclusion	12

PROJECT OVERVIEW

StudyTrack is a Python-based Student Academic Task & Performance Manager. It provides a centralized workflow for student records, assignments, marks, performance analysis and report generation.

KEY MODULES

- Student Management
- Assignment Management
- Marks / Performance Management
- Report Generation
- CSV Data Storage
- Validation and Error Handling

1. INTRODUCTION

In today's academic environment, students often struggle to manage their tasks, assignments and performance efficiently. Keeping track of deadlines, marks and overall progress can be challenging, especially without a centralized system.

The StudyTrack project aims to solve this problem by providing a simple, user-friendly and efficient Student Academic Task and Performance Manager. It helps students organize their academic life, track assignments, manage marks, and generate reports, all in one place.

2. PROBLEM STATEMENT

Students face difficulties in managing multiple assignments, tracking deadlines, and monitoring their academic performance. Existing solutions may be complex, not user-friendly, or lack essential features such as report generation and data visualization.

This project addresses these challenges by offering a centralized, minimal and easy-to-use system for academic management.

PROJECT NEED

- Centralized student information management.
- Simple assignment and task tracking.
- Consistent storage of marks and academic records.
- Quick generation of performance reports.
- Validation and error handling for reliable input.

3. OBJECTIVES

- To create a centralized platform for managing student data, assignments and marks.
- To help students stay organized and meet academic deadlines.
- To provide easy access to performance reports and analytics.
- To implement the project using Python with proper modular structure and best practices.

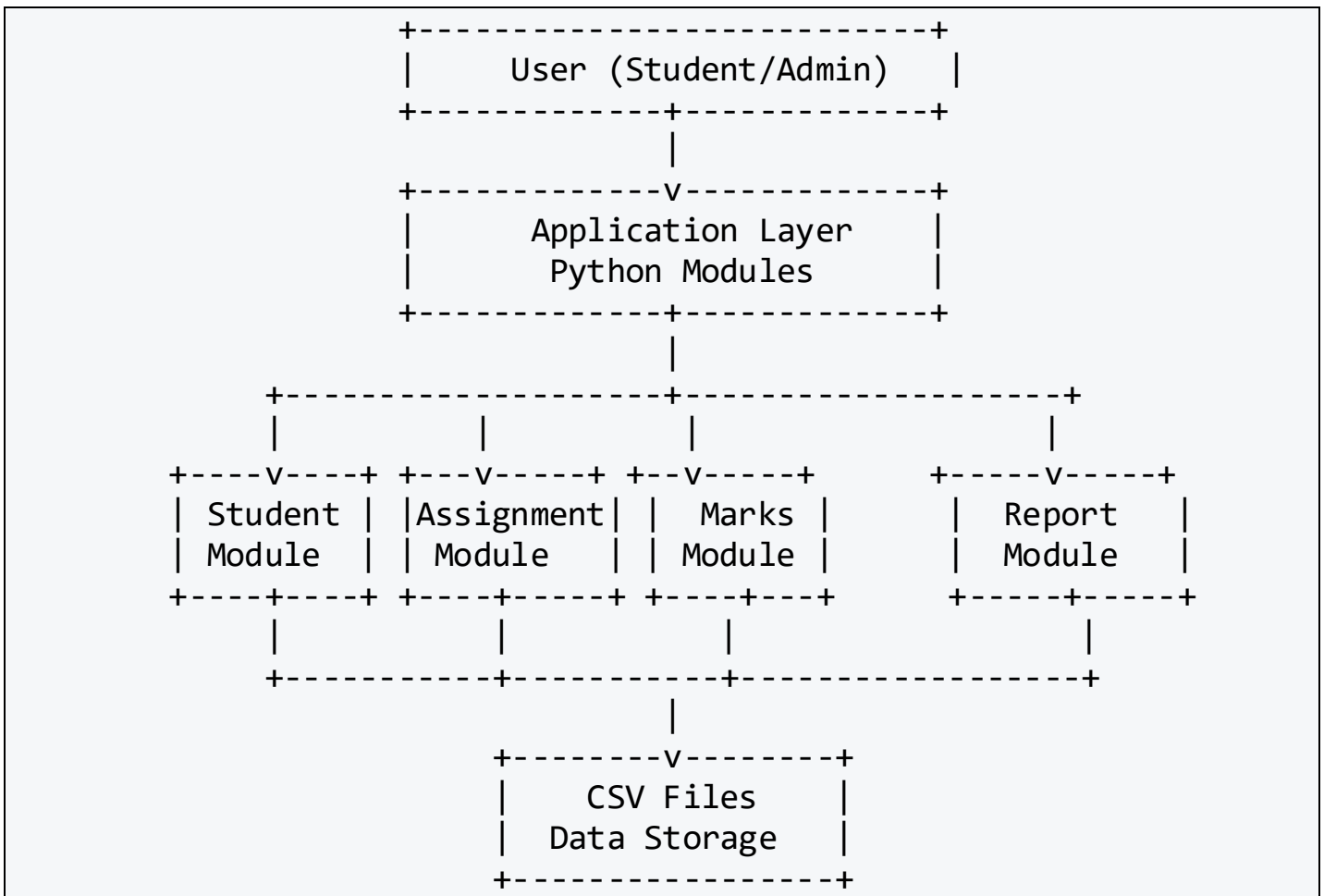
4. FUNCTIONAL REQUIREMENTS

Module	Main Functions	Expected Output
Student Management	Add, update, delete student details	Updated student records
Assignment Management	Create, track and manage assignments	Assignment/task status
Marks Management	Store and calculate marks, view performance	Marks and analytics
Report Generation	Generate individual and overall reports	Readable report
Data Storage	Read/write structured records	Persistent data

5. NON-FUNCTIONAL REQUIREMENTS

- Performance – Fast and responsive system.
- Usability – Simple and easy-to-use interface.
- Reliability – Controlled handling of invalid input and file errors.
- Maintainability – Modular and well-documented code.
- Error Handling – Clear messages instead of uncontrolled failures.

6. SYSTEM ARCHITECTURE

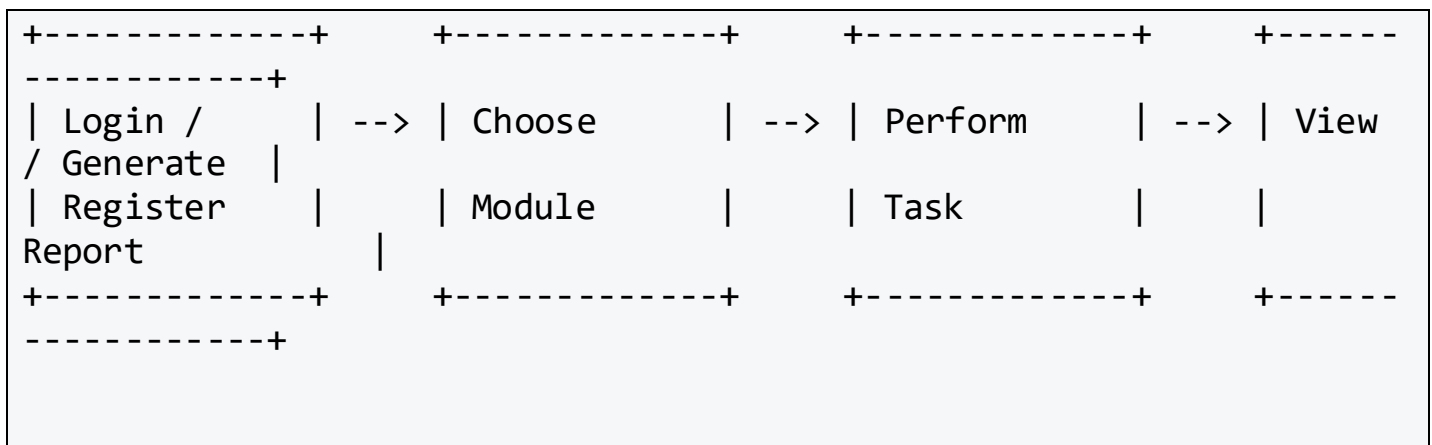


The system follows a modular architecture. Users interact with the application layer, which routes operations to student, assignment, marks and report modules. Data is persisted through structured CSV files.

ARCHITECTURE COMPONENTS

Layer	Components	Responsibility
User	Students / Admin	Provide input and view results
Application	Python modules	Coordinate workflow and business logic
Student	Student module	Manage student records
Assignment	Assignment module	Manage academic tasks
Marks	Marks module	Calculate performance
Report	Report module	Prepare summaries
Storage	CSV files	Persist records

7. PROCESS FLOW / WORKFLOW



WORKFLOW DESCRIPTION

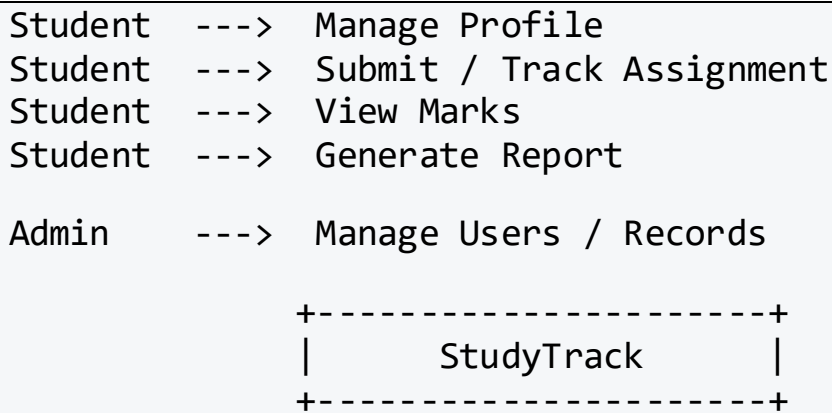
Step	Activity
1	User starts the application.
2	System loads required stored data.
3	User selects a module or operation.
4	System validates the entered data.
5	Selected operation is performed.
6	Records are updated when required.
7	System displays confirmation, result or error.

INPUT / OUTPUT EXAMPLES

Operation	Input	Output
Add Student	ID, name, course	Student record created
Add Assignment	Title, due date, status	Assignment stored
Marks	Student + subject marks	Performance summary
Report	Selected student/module	Readable report

8. UML DIAGRAMS

8.1 USE CASE DIAGRAM



8.2 CLASS DIAGRAM

Class / Component	Attributes	Operations
Student	id, name, course, email	login(), viewMarks(), submitAssignment()
Assignment	id, title, due_date, status	update(), delete()
Marks	student_id, subject, score	add(), update(), calculate()
Report	student_id, type	generate(), export()

8.3 MODULE RELATIONSHIP

```
main.py
|
+--> student.py
+--> assignment.py
+--> marks.py
+--> report.py
+--> data_handler.py
+--> utils.py
+--> CSV data files
```

9. IMPLEMENTATION

The project is developed in Python using a modular structure. Each major functionality is separated into a dedicated module so that changes can be made without unnecessarily affecting unrelated components.

KEY MODULES / FILES

No.	File / Module	Description
1	main.py	Entry point of the application
2	student.py	Student management module
3	assignment.py	Assignment management module
4	marks.py	Marks management module
5	report.py	Report generation module
6	data_handler.py	CSV read/write operations
7	utils.py	Helper and validation functions
8	test_project.py	Unit tests
9	config.py	Configuration settings

TECHNOLOGIES USED

- Python 3.x
- CSV data storage
- Python standard libraries
- Git / GitHub version control

10. DATA & STORAGE DESIGN

StudyTrack uses structured CSV files for lightweight local data storage. CSV is suitable for tabular student, assignment and marks information and keeps the project easy to inspect during academic evaluation.

File	Important Fields	Purpose
students.csv	student_id, name, course, email	Student records
assignments.csv	assignment_id, student_id, title, due_date, status	Assignment tracking
marks.csv	student_id, subject, score	Academic performance

SAMPLE RECORD

student_id,name,course,email 26MIP10068,Yashu Malviya,B.Tech CSE,yashu@example.com

DATA VALIDATION

- Required fields should not be empty.
- Student IDs should remain consistent.
- Marks should remain within the defined range.
- Assignment status should use accepted values.
- File operations should handle missing or invalid data safely.

DESIGN DECISIONS & RATIONALE

Decision	Rationale
Python	Matches the programming syllabus and supports rapid development.
CSV	Simple, readable and appropriate for a small academic dataset.
Modular files	Improves maintainability and separates responsibilities.
Validation	Reduces invalid records and runtime failures.
GitHub	Provides version history and repository submission.

11. TESTING

The project is tested using unit-level checks and manual functional testing. Test cases cover normal operations, invalid input, data validation and report generation.

Test Case	Input	Expected Output	Result
Add Student	Valid data	Student added	Pass
Add Assignment	Valid data	Assignment added	Pass
Invalid Input	Empty fields	Error message	Pass
Marks Calculation	Valid marks	Correct average	Pass
Search	Existing ID	Correct record	Pass
Search	Unknown ID	Not found message	Pass
Generate Report	Valid data	Report generated	Pass
CSV Storage	Save + reload	Data preserved	Pass

12. RESULTS

- Core student, assignment, marks and reporting features are implemented.
- CSV files store data correctly.
- Reports are generated in a readable format.
- Validation and error handling are included.
- Test cases cover valid and invalid scenarios.

13. CHALLENGES FACED

- Handling file data and validation.
- Implementing a clean modular structure.
- Generating reports with accurate data.
- Keeping student, assignment and marks records consistent.
- Writing and running tests.
- Organizing the project repository and report submission.

14. LEARNINGS

- Improved Python programming skills.
- Learned to work with files and CSV data.
- Gained experience in modular and reusable coding.
- Understood validation and error handling.
- Learned the importance of testing before submission.

15. FUTURE ENHANCEMENTS

- Add a web-based interface using Flask/Django.
- Integrate a database such as SQLite/MySQL.
- Add data visualization using charts/graphs.
- Implement role-based access.
- Add deadline notifications and richer report exports.

16. REFERENCES & CONCLUSION

REFERENCES

- Python Documentation – <https://docs.python.org/3/>
- VITyarthi – Build Your Own Project guidelines supplied for this project.
- GitHub Documentation – <https://docs.github.com/>
- Faculty-provided course material and programming practice resources.

CONCLUSION

StudyTrack demonstrates the practical use of Python programming concepts to solve an academic record-management problem. The system combines student management, assignment tracking, marks handling, report generation, CSV storage, validation and testing within a modular workflow. The project structure is designed to be understandable, maintainable and extendable. Future versions can introduce databases, graphical interfaces, dashboards, notifications and role-based access.

PROJECT SUBMISSION SUMMARY

Item	Description
Problem Understanding	Academic task and performance management problem
Functional Modules	Student, Assignment, Marks and Report
Non-Functional Requirements	Performance, usability, reliability, maintainability
Architecture	Modular Python application with CSV storage
Testing	Unit and manual test cases
Version Control	Git / GitHub repository
Documentation	12-page VIT-style project report

--- END OF REPORT ---